

delete_certificate

REFERENCE



API consumer



remove a certificate (or all of a type) from the reader's trust store

Overview

`delete_certificate` removes a certificate from the reader's trust store, identified by `certificateInfo.name`, `type`. It is **synchronous** — a successful delete returns `response.code 0` and the certificate is gone immediately (confirm with `get_installed_certificates`). It is a state-changing command on the device-management (MGMT) endpoint.

`delete_certificate` is destructive and irreversible — and omitting `certificateInfo.name` deletes every certificate of the given `type`. See the danger callout in [Notes and Caveats](#) before sending.

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	STATE-CHANGING (removes a certificate, or all certificates of a type)
Reversibility	IRREVERSIBLE (no undo; re-install with <code>install_certificate</code> to restore)
Required Request Fields	<code>command</code> , <code>requestId</code> , <code>certificateInfo</code> (with <code>type</code> ; <code>name</code> optional)
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	<code>get_installed_certificates</code> , <code>install_certificate</code> , <code>config_endpoint</code> , <code>set_wifi</code>

When to Use

Use `delete_certificate` to **clean up the reader's trust store**:

- **Remove an obsolete or expired certificate** — e.g. before rotating in a replacement.
- **Clear a slot before re-installing** — delete then `install_certificate` with the same `name`.
- **Wipe a whole type** — omit `name` to remove every certificate of a `type` (e.g. all `wifi` certificates) in one call. Use deliberately.

Delete only certificates that are **not in active use**. On a TLS endpoint, deleting the broker/client certificate the reader is currently connected with will break that connection.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

`delete_certificate` identifies certificates by `name` / `type` only, so it carries **no credential field** — there is nothing to mask. The `requestId` you send is echoed in the response so you can pair reply to request.

See [Topics and Channels](#).

Key Fields

- `certificateInfo.type` (request) — the certificate category (`mqtt` , `wifi` , `filestore` , `client` , `server`); required. Confirm exact names/types with `get_installed_certificates` first.
- `certificateInfo.name` (request) — the certificate to delete. ⚠️ **Omitting it deletes every certificate of type** (bulk). Verify the exact name with `get_installed_certificates` (installed entries are named `{name}_{component}`).
- `response.code` (response) — read this to determine success vs failure: `0` (deleted), `21` (no such certificate). See [Response Codes](#).

Request Payload

The envelope is `command` , `requestId` , and the `certificateInfo` object naming the certificate to delete. Pick a **certificate type** below; the Example and Schema update together. Use the **Schema** tab for the full field reference and the **Example** tab for a single-certificate delete.

Certificate scenario

Delete wifi ca cert

Request payload

Example: wifi / ca_cert

```
1 {
2   "command" : "delete_certificate" ,
3   "requestId" : "abc123" ,
4   "certificateInfo" : {
5     "name" : "ca_cert" ,
6     "type" : "wifi"
7   }
8 }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	delete_certificate	Fixed command name.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
2	requestId *	string	—	abc123	A unique identifier for the request; echoed back in the response.
3	certificateInfo *	object	—	—	Identifies the certificate(s) to delete. Required by the contract, and the device rejects a request without it with code 101 .
4	name	string	—	test_soti	⚠ The certificate name to delete. If omitted, ALL certificates of the given type are deleted (bulk, destructive — see Notes).
5	type *	enum	server client mqtt wifi filestore	client	Certificate category. Required within certificateInfo . Selects the certificate (with name) or the whole set (without name).

To delete **every certificate of a type**, omit `name` and send only `type` — a destructive bulk operation:

Request — bulk delete (all wifi certificates)

```

1  {
2    "command" : "delete_certificate" ,
3    "requestId" : "abc123" ,
4    "certificateInfo" : {
5      "type" : "wifi"
6    }
7  }
```

Request Semantics

- `certificateInfo` is required, and so is the `type` inside it. The contract requires the payload outright, and the device agrees — a request without it is rejected with code `101`. Always send `certificateInfo` with at least a valid `type`.
- Type-`null` certificates cannot be deleted.** `delete_certificate` matches on `type`, so a certificate that `install_certificate` created **without a `type`** (`type: null`, which the installer accepts) matches no `type` value and returns code `21` for every attempt — it becomes un-deletable through this command. Avoid the situation by always sending `type` to `install_certificate`; to clear an existing orphan, re-install the same `name` with an explicit `type`.

Response Payload

`delete_certificate` returns a **status-only** envelope — `command`, `requestId`, `apiVersion`, and `response`. There is no certificate echo; confirm the removal with `get_installed_certificates`.

Response payload

```
1  {
2    "command" : "delete_certificate" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "code" : 0 ,
7      "description" : "Success"
8    }
9  }
```

Response payload

★ required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command ★	string	—	delete_certificate	Echoes the executed command.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	response *	object	—	—	Status-only result — delete_certificate echoes no certificate data.
5	code *	integer	0–101	0	0 = Success; non-zero = error. Required by the contract. See Response Codes below.
6	description *	string	—	Success	Human-readable status.

Response Codes

`delete_certificate` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the matching certificate (or all of the type) was removed.
9	File not found (<code>IOT_STATUS_FILE_NOT_FOUND</code>) — a backing certificate file could not be found.
21	Certificate not found (<code>IOT_ERROR_CERT_NOT_FOUND</code>) — the <code>name</code> / <code>type</code> is not a saved certificate. See the not-found note below.
101	Default error — generic catch-all (e.g. a malformed payload; see the note below).

Two live-observed behaviors to code against: (1) **A not-found delete returns code 21 but with the description text "File not found"** (the wording of code 9) — key on the numeric 21 , not the string. (2) A **malformed payload** — a missing required `type` , or a missing `certificateInfo` — returns the generic 101 ("Default error"), while an **invalid type** value returns 23 ("Invalid enum value"). Gate success strictly on `code == 0`.

Examples

Publish `delete_certificate` naming the certificate to remove; the device replies with the status-only envelope under [Response Payload](#). The bulk-delete request (omit `name`) is shown under [Request Payload](#).

Targeting a `name` / `type` that is **not installed** returns **code 21** (note the **"File not found"** description):

Response — certificate not found (code 21)

```
1  {
2    "command" : "delete_certificate" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "code" : 21 ,
7      "description" : "File not found"
8    }
9  }
```

Notes and Caveats

 **IRREVERSIBLE — AND OMITTING `name` WIPES EVERY CERTIFICATE OF THE TYPE**

There is no undo and no confirmation prompt. A `delete_certificate` with `certificateInfo.type` but no `name` deletes **every** certificate of that type. Always include an explicit `name` unless a full type-wipe is intended, and verify the exact name first with `get_installed_certificates`. Never delete a certificate an **active TLS connection** relies on — doing so breaks that connection. Recovery in every case means re-installing with `install_certificate`.

Because the response is status-only, **verify the result with `get_installed_certificates`** — confirm the deleted `name` / `type` no longer appears. For the concept-level view, see The Security Model and Install and Manage Certificates.

Related:  The Security Model ·  `get_installed_certificates` · 
`install_certificate` ·  `config_endpoint` ·  [Response Codes](#)